Phase 6: 模型搭建实现指导

1. Scaled Dot-Product Attention

1.1 关键步骤与形状追踪

```
输入:
  Q: (..., n, d_k)      — n 个 query
  K: (..., m, d_k)      — m 个 key
  V: (..., m, d_v)      — m 个 value
  mask: (..., n, m)      — 可选

Step 1: scores = Q @ K^T / sqrt(d_k)
  Q:              (..., n, d_k)
  K^T:            (..., d_k, m)    ← K.transpose(-2, -1)
  Q @ K^T:        (..., n, m)
  / sqrt(d_k):    标量除法

Step 2: 应用 mask
  mask 为 False 的位置: scores 设为 -inf（或 float('-inf')）

Step 3: softmax(scores, dim=-1)
  结果:           (..., n, m)      — 每个 query 对所有 key 的权重

Step 4: weights @ V
  weights:        (..., n, m)
  V:              (..., m, d_v)
  output:         (..., n, d_v)
```

1.2 mask 应用的技巧

```
# 方法 1: 用 masked_fill_
scores = scores.masked_fill(~mask, float('-inf'))

# 方法 2: 用 where
scores = torch.where(mask, scores, torch.tensor(float('-inf')))

# 方法 3: 直接加
scores = scores + (~mask) * float('-inf')
# 注意: False * -inf = nan，所以需要用 (~mask).float() * (-1e9) 或 masked_fill
```

推荐使用 `masked_fill`，最干净。

1.3 测试中的 mask 形状

- 3D 测试: Q(4, 12, 64), K(4, 16, 64), mask(4, 12, 16)
- 4D 测试: Q(2, 2, 12, 64), K(2, 2, 16, 64), mask(2, 2, 12, 16)
- 你的实现应对任意 `...` batch 维度工作

1.4 sqrt(d_k) 的获取

```
import math
d_k = Q.shape[-1]
scale = math.sqrt(d_k)
# 或 scale = d_k ** 0.5
```

2. RoPE 实现

2.1 预计算频率

```
对于 i = 0, 1, ..., d_k/2 - 1:
  freq_i = 1.0 / (theta ^ (2i / d_k))

对于 pos = 0, 1, ..., max_seq_len - 1:
  angle[pos][i] = pos * freq_i

cos_cached[pos][i] = cos(angle[pos][i])
sin_cached[pos][i] = sin(angle[pos][i])
```

2.2 PyTorch 实现的关键操作

```
# 频率向量
freqs = 1.0 / (theta ** (torch.arange(0, d_k, 2).float() / d_k))
# 形状: (d_k/2,)

# 位置向量
positions = torch.arange(max_seq_len).float()
# 形状: (max_seq_len,)

# 外积得到角度矩阵
angles = torch.outer(positions, freqs)
# 形状: (max_seq_len, d_k/2)

# 预计算 cos 和 sin
cos_cached = torch.cos(angles)  # (max_seq_len, d_k/2)
sin_cached = torch.sin(angles)  # (max_seq_len, d_k/2)
```

2.3 应用旋转

输入 x 的形状为 `(..., seq_len, d_k)` ，需要对偶数和奇数维度做旋转。

```
对于每个维度对 (x_{2i}, x_{2i+1}):
  x_{2i}'   = x_{2i} * cos - x_{2i+1} * sin
  x_{2i+1}' = x_{2i} * sin + x_{2i+1} * cos
```

拆分偶数/奇数维度的方法：

```
# 方法 A: 切片
x_even = x[..., 0::2]   # (..., seq_len, d_k/2)  偶数索引
x_odd  = x[..., 1::2]   # (..., seq_len, d_k/2)  奇数索引

# 方法 B: reshape 后拆分
x_paired = x.reshape(*x.shape[:-1], -1, 2)  # (..., seq_len, d_k/2, 2)
x_even = x_paired[..., 0]
x_odd  = x_paired[..., 1]

# 方法 C: 用复数（更优雅但可能不够直觉）
x_complex = torch.view_as_complex(x.reshape(*x.shape[:-1], -1, 2))
# 旋转 = 乘以 e^(jθ)
```

2.4 用 token_positions 索引

```
# token_positions: (..., seq_len)  包含每个 token 的位置索引
# cos_cached: (max_seq_len, d_k/2)

# 需要从 cos_cached 中按 token_positions 取出对应行
# cos_cached[token_positions] → (..., seq_len, d_k/2)
cos = cos_cached[token_positions]
sin = sin_cached[token_positions]
```

2.5 重新拼接

```
# 旋转后需要把偶数和奇数维度交错回去
# 方法 A: torch.stack + reshape
result = torch.stack([x_even_rotated, x_odd_rotated], dim=-1)
result = result.reshape(*x.shape)

# 方法 B: 创建空张量，填入
result = torch.empty_like(x)
result[..., 0::2] = x_even_rotated
result[..., 1::2] = x_odd_rotated
```

3. Multi-Head Self-Attention

3.1 形状变换流程

```
输入 x: (B, T, D)        D = H * d_k

1. 投影:
   Q = x @ W_Q^T         (B, T, D)
   K = x @ W_K^T         (B, T, D)
   V = x @ W_V^T         (B, T, D)

2. 拆分为多头:
   Q → (B, T, H, d_k) → transpose(1,2) → (B, H, T, d_k)
   K → (B, T, H, d_k) → transpose(1,2) → (B, H, T, d_k)
   V → (B, T, H, d_k) → transpose(1,2) → (B, H, T, d_k)

3. 应用 RoPE（只对 Q 和 K）:
   Q = rope(Q, positions)    需要 positions: (B, T) 或 (1, T)
   K = rope(K, positions)

4. SDPA:
   attn_out = sdpa(Q, K, V, causal_mask)   (B, H, T, d_k)

5. 拼接:
   attn_out → transpose(1,2) → (B, T, H, d_k) → reshape → (B, T, D)

6. 输出投影:
   output = attn_out @ W_O^T              (B, T, D)
```

3.2 因果 mask 的构造

```python
# 对于 self-attention，n = m = T
T = x.shape[1]  # 或用 sequence_length

# 下三角矩阵: mask[i][j] = True if j <= i
causal_mask = torch.tril(torch.ones(T, T, dtype=torch.bool, device=x.device))

# 或
causal_mask = torch.ones(T, T, dtype=torch.bool, device=x.device)
causal_mask = torch.triu(causal_mask, diagonal=1).logical_not()
```

3.3 注意事项

- **不带 RoPE 的版本** (`run_multihead_self_attention`)：测试中也需要因果 mask
- **带 RoPE 的版本**：token_positions 可能是 None，此时默认 `[0, 1, ..., T-1]`
- **位置张量形状**：在 RoPE 中，head 维度被当作 batch 维度处理
  - 传给 RoPE 的 Q 形状: (B, H, T, d_k)，positions: (B, T) 或 (1, T)
  - RoPE 需要支持 positions 被广播到 (B, H, T) — 通常自动广播即可

3.4 reshape 拆分多头的细节

```
# W_Q 的形状: (D, D) = (H*d_k, D)
# Q = x @ W_Q^T  → (B, T, D)
# 需要 reshape 为 (B, T, H, d_k) 然后 transpose 为 (B, H, T, d_k)

# 关键: W_Q 的前 d_k 行对应 head 0，接下来 d_k 行对应 head 1，...
# 所以 Q[:, :, :d_k] 对应 head 0，这与 reshape(B, T, H, d_k) 的语义一致
Q = Q.view(B, T, H, dk).transpose(1, 2)  # (B, H, T, dk)
```

4. SwiGLU FFN

4.1 形状追踪

```
输入 x: (..., d_model)

gate = SiLU(W1 @ x)     (..., d_ff)     W1: (d_ff, d_model)
value = W3 @ x           (..., d_ff)     W3: (d_ff, d_model)
hidden = gate ⊙ value    (..., d_ff)     逐元素乘法
output = W2 @ hidden     (..., d_model)  W2: (d_model, d_ff)
```

4.2 d_ff 的计算

```
# d_ff ≈ 8/3 * d_model，取 64 的倍数
d_ff = int(8 / 3 * d_model)
d_ff = ((d_ff + 63) // 64) * 64  # 向上取到 64 的倍数

# 但在测试中，d_ff 是直接传入的（128），不需要你计算
# 只有在构建完整 TransformerLM 时才需要按公式计算
```

5. Transformer Block

5.1 forward 伪代码

```
def forward(self, x):
    # 子层 1: attention
    residual = x
    x = rmsnorm_1(x)
    x = multihead_attention(x)     # 内含 RoPE 和因果 mask
    x = residual + x

    # 子层 2: FFN
    residual = x
    x = rmsnorm_2(x)
    x = swiglu_ffn(x)
    x = residual + x

    return x
```

5.2 token_positions 的传递

- TransformerBlock 内的 MHA 需要 token_positions 来做 RoPE
- 你可以在 MHA 内部默认生成 positions（当不传入时）
- 或者在 TransformerBlock.forward 中接收 positions 并传递给 MHA
- 在 adapter 中，positions 来自 `torch.arange(seq_len)`

6. Transformer LM

6.1 forward 伪代码

```
def forward(self, token_ids):
    # token_ids: (B, T)

    x = embedding(token_ids)          # (B, T, D)

    # 构造 positions（如果不在外部传入）
    positions = torch.arange(T, device=x.device).unsqueeze(0)  # (1, T)

    for layer in self.layers:
        x = layer(x, positions)       # (B, T, D)  每层传入相同的 positions

    x = ln_final(x)                   # (B, T, D)
    logits = lm_head(x)               # (B, T, V)

    return logits
```

6.2 context_length 参数

- `context_length` 决定了 RoPE 预计算的最大序列长度（max_seq_len）
- 实际输入序列可以短于 context_length（测试 `test_transformer_lm_truncated_input` 验证了这一点）
- 但不应超过 context_length

6.3 adapter 中 load_state_dict 的使用

```
# 在 run_transformer_lm adapter 中:
model = TransformerLM(vocab_size, context_length, d_model, num_layers, num_heads, d_ff,
model.load_state_dict(weights)  # weights 的 key 必须与 model 的 state_dict key 匹配
output = model(in_indices)
return output
```

如果 key 不匹配， `load_state_dict` 会报错。用 `strict=False` 可以忽略不匹配，但这不是你想要的——你应该让 key 完全匹配。

7. 调试清单

实现每个模块后，按这个顺序检查：

1. **形状检查**：在 forward 中加临时 assert 检查中间张量形状
2. **state_dict 检查**：`print(model.state_dict().keys())` 确认 key 名
3. **数值检查**：用小规模输入手动验证几步计算
4. **测试**：运行对应的 `uv run pytest -k test_xxx -v`

常见错误：

- transpose 后忘记 contiguous（某些操作需要连续内存）
- 因果 mask 没有正确应用（维度不匹配或布尔值反了）
- RoPE 的 sin/cos 维度与 x 不匹配（忘记处理 head 维度作为 batch 维度）
- W 矩阵形状搞反（`(d_out, d_in)` vs `(d_in, d_out)`）